

EXPERIMENT NO: 08 - Steganography Analysis and Hidden Data Detection using StegExpose

DATE:

1. AIM

To conduct a comprehensive steganalysis lifecycle on digital image evidence using the StegExpose command-line utility; to evaluate Least Significant Bit (LSB) statistical properties and detection algorithms across a target evidence directory; to compute quantitative suspicion scores based on configurable thresholds; and to automatically compile, format, and export the analytical findings into a structured CSV report for forensic documentation.

2. SOFTWARE / TOOLS REQUIRED

- **Operating System:** Windows, Linux, or macOS
- **Runtime Environment:** Java Runtime Environment (JRE)
- **Forensics Tool:** StegExpose (Java .jar executable)
- **Evidence Files:** Target image files (e.g., .png, .jpg, .bmp)

3. DESCRIPTION

StegExpose is a specialized steganalysis tool designed to detect the presence of hidden data within digital images. Unlike traditional steganography tools used to embed data, StegExpose serves a purely investigative role. It evaluates the statistical properties of an image to identify mathematical anomalies indicative of data hiding techniques, particularly Least Significant Bit (LSB) steganography.

The tool processes images and outputs a quantitative "suspect score" ranging from 0 to 1. By comparing this score against established thresholds, forensic investigators can mathematically estimate the likelihood that an image contains clandestine data, allowing them to triage suspicious files rapidly without needing the original extraction password.

4. PROCEDURE

Step 1: Open Command Prompt as Administrator

- Press the Windows Key, type cmd, right-click on **Command Prompt**, and select **Run as administrator**.

Step 2: Navigate to the Working Directory

- Use the change directory (cd) command to point the terminal to your project folder where StegExpose.jar and your evidence directory are located:
- And check if java is available.

DOS

java --version

(it shows jre installed or not)

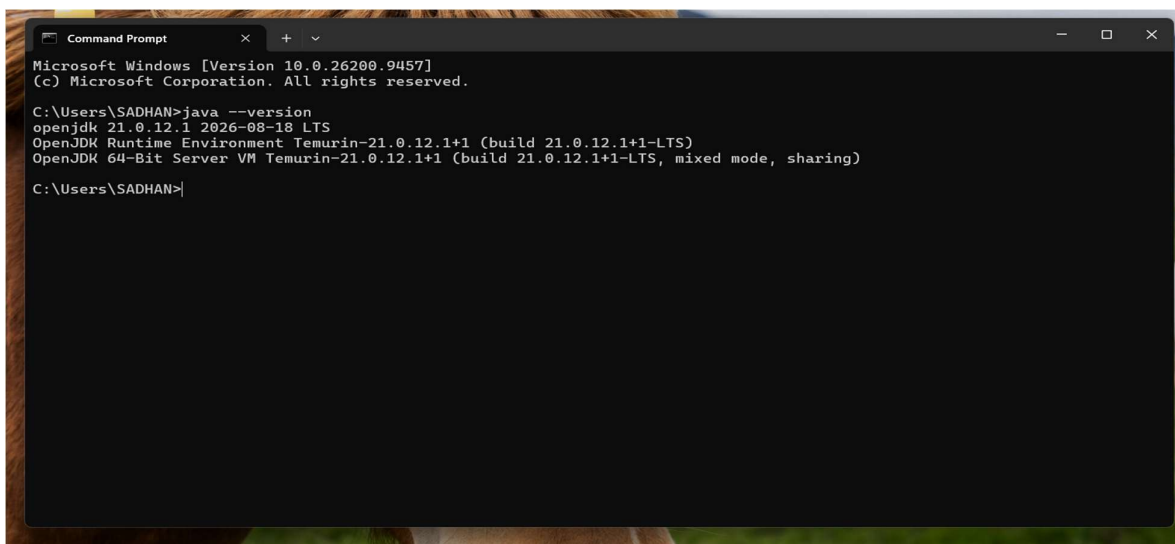
A screenshot of a Windows Command Prompt window. The title bar reads 'Command Prompt'. The text inside the window shows the command 'C:\Users\SADHAN>java --version' and its output: 'openjdk 21.0.12.1 2026-08-18 LTS', 'OpenJDK Runtime Environment Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS)', and 'OpenJDK 64-Bit Server VM Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS, mixed mode, sharing)'. The prompt 'C:\Users\SADHAN>' is visible at the bottom.

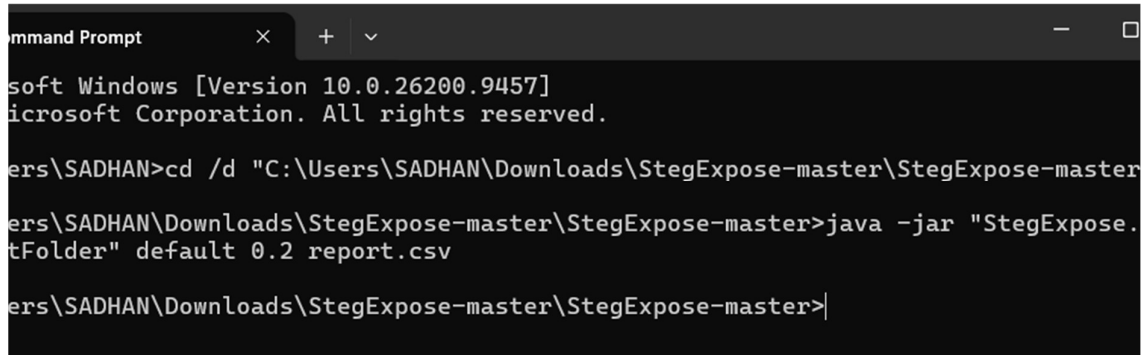
Figure 1: Navigating to the working directory in the Command Prompt

Step 3: Execute StegExpose Analysis with CSV Output

- Run the StegExpose command, passing the path to your testFolder, the execution speed parameter (default), the statistical threshold (0.2), and the desired output report filename (report.csv):

DOS

```
java -jar "StegExpose.jar" "testFolder" default 0.2 report.csv
```



```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\SADHAN>cd /d "C:\Users\SADHAN\Downloads\StegExpose-master\StegExpose-master"
C:\Users\SADHAN\Downloads\StegExpose-master\StegExpose-master>java -jar "StegExpose.jar" "testFolder" default 0.2 report.csv
C:\Users\SADHAN\Downloads\StegExpose-master\StegExpose-master>
```

Figure 2: Executing StegExpose via CLI and redirecting output to report.csv

Step 4: Verify and Review the Generated Report

- Once the execution finishes, navigate to your folder or open the file in Microsoft Excel/Notepad to verify the generated report.csv contents.
- Otherwise, directly open the csv file on excel.

DOS

```
notepad report.csv
```

	A	B	C	D	E	F	G	H
1								
2	File name	Above stego threshold?	Secret message size in bytes (	Primary Se	Chi Square	Sample Pa	RS analysis	Fusion (mean
3	clean_7221186570_82a10bc040_z.png	FALSE	13416	0.113292	0.142357	null	0.048735	0.101462
4	clean_7222762386_2688184c8e.png	FALSE	7391	NaN	3.74E-11	0.240066	0.258613	0.166226
5	clean_7228374452_8f743dc3d4_z.png	FALSE	2449	0.031137	0.007355	null	0.016071	0.018188
6	clean_7228718722_1cf25dff3e_z.png	FALSE	14194	0.116141	0.002102	null	0.099881	0.072708
7	clean_7231623538_f51db2a35a_z.png	FALSE	8198	0.050938	0.018913	null	0.061653	0.043834
8	clean_7232193260_c2fd8c0f25_z.png	FALSE	22569	0.257388	0.002651	0.096921	0.106751	0.115928
9	clean_7232202430_d38bd6d6986_z.png	FALSE	25126	NaN	0.009698	0.180171	0.21014	0.133336
10	clean_7232206610_b8cfded120_z.png	FALSE	10190	0.02848	0.035967	0.08053	0.084405	0.057346
11	clean_7232220662_3d42c69109_z.png	FALSE	18000	NaN	0.012527	0.132871	0.147407	0.097602
12	clean_7234473324_bb8a82b5bd.png	FALSE	1403	0.01654	0.002058	null	0.036617	0.018405
13	clean_7235558256_3099066753.png	FALSE	21450	0.175461	0.176949	null	0.201671	0.184694
14	clean_7235972310_8c25258da5.png	FALSE	10882	0.184965	0.005157	null	0.097284	0.095802
15	clean_7237798856_86870c8823_z.png	FALSE	3654	0.073302	8.57E-04	null	0.069658	0.047939
16	stego_6666458261_e455d262b5_z.png	TRUE	114785	NaN	0.06767	0.72929	0.738013	0.511658
17	stego_6672108499_85c582a7f9.png	TRUE	137047	NaN	0.999991	null	0.869877	0.934934
18	stego_6672542201_532f70bffe.png	TRUE	67141	NaN	0.093713	0.781242	0.768072	0.547675
19								

Figure 3: Opening and reviewing the generated CSV report detailing LSB scores and classifications

5. OBSERVATIONS

- **Batch Processing:** StegExpose scanned all image files within the target evidence folder sequentially.
- **Classification Metrics:** Each file was assigned an LSB score evaluated against the configured 0.2 threshold. Files exceeding the limit were flagged as suspected steganography carriers.
- **Report Compilation:** The console results were successfully written to report.csv, containing structured columns for filenames, scores, and status flags.

RUBRICS

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

6. RESULT

The steganography detection experiment was successfully executed using StegExpose. The tool scanned the testFolder directory via the command line, analyzed combinations of LSB metrics, and generated a clean CSV report detailing the cleanliness or suspicion level of each digital image artifact.